

Описание архитектуры проекта

Название проекта: **Marketing Service (Web API)** — веб-сервис управления маркетинговыми кампаниями с использованием большой языковой модели (LLM): классификация сообщений, парсинг описания кампании в структуру, разбиение кампании на подзадачи (по подходящему шаблону или свободно через LLM), ответы на вопросы по данным кампаний, отслеживание прогресса выполнения и память диалога с пользователем. Изначально проект был Telegram-ботом, сейчас это REST API + веб-чат с боковой панелью кампаний и шаблонов.

Список ключевых технологических компонентов системы

Веб-бэкенд (REST API)

- FastAPI + Uvicorn (ASGI), автодокументация OpenAPI
- Эндпоинты чата, истории диалога, кампаний, шаблонов, участников, напоминаний
- Раздача статики веб-чата

LLM-компонент

- Облачный доступ к модели через OpenRouter (`openai/gpt-5-nano`)
- Классификация входящего сообщения (кампания / вопрос / прочее)
- Парсинг свободного текста в структуру кампании
- Подбор подходящего шаблона разбиения
- Разбиение задачи на подзадачи (когда шаблон не подходит)
- Ответы на вопросы пользователя по контексту кампаний (Q&A) с учётом истории диалога

База данных

- SQLite + SQLAlchemy (асинхронно, через `aiosqlite`)
- Кампании и подзадачи (с отметками выполнения)
- Шаблоны кампаний и их подзадачи
- Участники чата (имя + роль)
- Ожидающие подтверждения («pending») кампании
- История сообщений чата (память диалога)

Бизнес-логика

- Чистый слой `core/service.py` без привязки к транспорту
- Детерминированное построение подзадач из шаблона (бюджет — % от общего, срок — доля дней до дедлайна)
- Сопоставление участников и ролей, разрешение само-упоминаний («я», «дизайнер (я)»)
- Дозаполнение и слияние данных кампании при уточняющем диалоге
- Авто-пересчёт статуса кампании по состоянию её подзадач

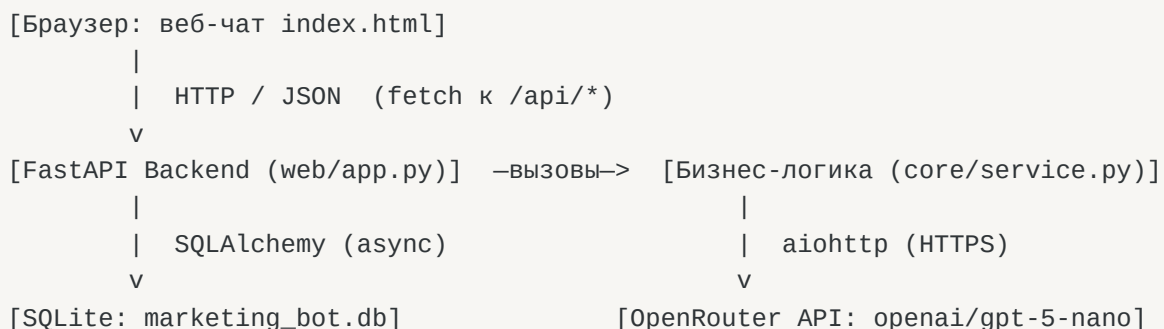
Интерфейс

- Статический веб-чат (HTML/CSS/JS, без сборщика)
- Боковая панель: список кампаний с подзадачами, шаблоны, дедлайны на сегодня
- Чекбоксы выполнения подзадач и кнопка отметки всей кампании ✓ завершена
- Восстановление истории переписки при входе и кнопка «очистить чат»
- Логин по имени и роли (хранится в браузере, без паролей)
- Формы добавления/редактирования шаблонов

Технологический стек системы управления маркетинговыми кампаниями

1. Общая архитектура

Система построена как **монолитное веб-приложение**: единый бэкенд на FastAPI, одна встроенная реляционная БД SQLite, обращение к LLM через облачный шлюз OpenRouter и лёгкий статический фронтенд (один HTML-файл). Все серверные компоненты запускаются одним процессом Uvicorn; внешних сервисов и оркестрации не требуется.



2. Бэкенд: FastAPI + Uvicorn

- **Роль:** единый сервис — приём сообщений чата, CRUD кампаний/шаблонов/участников, история диалога, обращение к LLM, выдача статистики веб-чата.
- **Почему:** асинхронный, минимум кода, встроенная документация OpenAPI (`/docs`), просто стыкуется со статическим фронтендом и с HTTP-клиентом к LLM.
- **Основные эндпоинты:**
 - `POST /api/chat` — отправить сообщение; ответ: `question` / `other` (текст), `campaign` (превью + `pending_id`) или `clarify` (не хватает данных — список `missing`).
Сохраняет и реплику пользователя, и ответ бота в историю.
 - `POST /api/campaigns/refine` — дозаполнить ожидающую кампанию новым сообщением
 - `POST /api/campaigns/confirm` — подтвердить создание кампании по `pending_id`
 - `POST /api/campaigns/cancel` — отменить ожидающую кампанию
 - `GET /api/campaigns`, `DELETE /api/campaigns/{name}` — список и удаление кампаний
 - `PATCH /api/campaigns/{name}/completed` — отметить всю кампанию выполненной (или вернуть в работу)
 - `PATCH /api/campaigns/{name}/subtasks/{id}` — переключить статус подзадачи; кампания автоматически помечается завершённой, если все её подзадачи выполнены
 - `GET/POST/PUT/DELETE /api/templates[/]{name}]` — CRUD шаблонов
 - `GET/POST/DELETE /api/users[/]{name}]` — реестр участников чата
 - `GET /api/reminders/today` — подзадачи с дедлайном сегодня (отфильтрованы от уже выполненных)
 - `GET /api/chat/history`, `DELETE /api/chat/history` — загрузка и очистка истории диалога пользователя
 - `GET /api/health` — health-check (`{"status":"ok"}`)
 - `GET /` и `/static/*` — веб-чат и его статика
- **Особенности:**
 - Время жизни приложения (`lifespan`): создание таблиц, мини-миграции SQLite, инициализация встроенных шаблонов, фоновая задача очистки просроченных pending-кампаний (раз в час).
 - Таймаут на вызов LLM — 60 секунд; ошибки LLM мапятся в HTTP 502 с понятным текстом.
 - Валидация входных данных через Pydantic-схемы; ошибки бизнес-логики (`CampaignError`) → HTTP 400/404.

3. База данных: SQLite + SQLAlchemy

- **СУБД:** SQLite (файл `marketing_bot.db`), доступ асинхронный — SQLAlchemy 2.0 + `aiosqlite`.
- **Почему:** нулевая настройка, один файл, достаточно для MVP (десятки–сотни кампаний); при необходимости легко заменить на PostgreSQL, вынеся `DATABASE_URL` в окружение.
- **Схема** (упрощённо):

```
campaigns(id, name UNIQUE, responsible, deadline, budget_rub,
          chat_id, created_by, template_used, completed_at)
subtasks(id, campaign_id -> campaigns.id, name, responsible,
          budget_rub, execution_days, deadline, reminder_sent,
          completed_at)
campaign_templates(id, name UNIQUE, description, total_budget_rub,
                  participants_example, deadline_example, created_by)
template_subtasks(id, template_id -> campaign_templates.id, name,
                  responsible_role, budget_percentage, execution_days_ratio)
chat_users(id, name UNIQUE, role, created_at, updated_at)
chat_messages(id, user_name INDEX, role, content, created_at INDEX)
pending_campaigns(id UUID, data JSON, created_at)
```

- **Миграции:** `Base.metadata.create_all` создаёт недостающие таблицы; недостающие колонки на старой БД добавляются вручную (`_migrate_sqlite`, `ALTER TABLE`) — в т.ч. `completed_at` у `campaigns` и `subtasks` при апгрейде.
- **pending-кампании:** распарсенные, но ещё не подтверждённые кампании хранятся в БД (а не в памяти процесса) — переживают перезапуск, работают при нескольких воркерах, автоматически удаляются через `PENDING_TTL_HOURS = 24` ч.
- **chat_messages:** единая таблица истории диалога. Ключ диалога — `user_name` (или `_anonymous_`, если имя не задано); хранятся и реплики пользователя, и ответы бота. Используется для двух целей: восстановления переписки в UI и подмешивания последних N реплик в контекст LLM.

4. LLM-компонент: OpenRouter (openai/gpt-5-nano)

- **Доступ:** облачный — OpenRouter, OpenAI-совместимый эндпоинт `https://openrouter.ai/api/v1/chat/completions`, модель `openai/gpt-5-nano`.
- **Интеграция:** из бэкенда асинхронным HTTP-клиентом (`aiohttp`), ключ — переменная окружения `OPENROUTER_API_KEY`.
- **Сценарии вызова** (класс `NemotronLLM`, модуль `ml/llm.py`):
 - `classify_message` — одно слово: `campaign` / `question` / `other` (`temperature=0`)

- `parse_campaign` — текст → JSON: `campaign_name`, `task`, `deadline`, `participants`, `budget_rub`, `confidence_score`; в промпт подаётся реестр участников и текущая дата
- `pick_template` — выбор имени шаблона из списка или `none`
- `decompose_task_llm` — JSON со списком подзадач (имя, ответственный, дни, бюджет) с жёсткими ограничениями
- `answer_question` — краткий ответ по собранному контексту всех кампаний и недавним репликам диалога; собирает полноценный `messages`-массив (system + история до 10 реплик + текущий вопрос), `temperature=0.2`
- **Парсинг ответа:** `_extract_json` снимает markdown-ограждение и обрезает по фигурным скобкам — устойчиво к «болтливости» модели.
- **Промпты** вынесены в `ml/prompts.py` (русский язык, формат «только JSON»); для Q&A используется `QA_SYSTEM` — system-промпт со встроенным контекстом кампаний и явными правилами учёта предыдущих реплик («а у этой?», «а кто там?»).
Нормализация дат — через `dateparser` (порядок ДМГ).

5. Шаблоны кампаний и разбиение на подзадачи

- **Встроенные шаблоны** (`config/templates.py`): «Таргет», «Контекст», «Запуск продукта» — каждый описывает подзадачи с долей бюджета (`budget_percentage`) и долей срока (`execution_days_ratio`). При старте записываются в БД, если их там ещё нет; пользователь может добавлять/редактировать свои шаблоны через API и веб-форму.
- **Логика разбиения** (`core/service.py`):
 1. LLM выбирает подходящий шаблон по названию/задаче кампании.
 2. Если шаблон выбран и участников хватает на его роли — подзадачи строятся **детерминированно**: бюджет = % от общего, срок = доля от дней до дедлайна, ответственные раздаются по кругу с указанием роли.
 3. Иначе — **свободное разбиение через LLM** под реально указанную команду (с примерами из шаблонов в подсказке).
 4. Имя применённого шаблона сохраняется в `Campaign.template_used` (или `null`).
- Подзадачи получают целочисленный срок `execution_days ≥ 1` и дату дедлайна (`today + execution_days`); суммы бюджетов и сроков ограничены параметрами кампании.

6. Отслеживание прогресса (новое)

- **Модель:** у `Campaign` и `Subtask` есть колонка `completed_at` (timestamp или `null`).
Признак «выполнено» — это `completed_at IS NOT NULL`; в API отдаются производные поля `completed: bool` и `completed_at: iso`.

- **Эндпоинты:**

- `PATCH /api/campaigns/{name}/subtasks/{id}` — снять/поставить отметку выполнения подзадачи.
- `PATCH /api/campaigns/{name}/completed` — отметить кампанию вручную; при `completed=true` заодно проставляет все её подзадачи.
- **Авто-пересчёт** (`_recompute_campaign_completion`): после переключения подзадачи в той же транзакции пересчитывается состояние кампании — если все подзадачи выполнены, кампания помечается завершённой; если снять отметку хотя бы с одной, кампания возвращается «в работу».
- **UI:** чекбокс в строке подзадачи, кнопка «✓ Отметить всю кампанию выполненной» / «🔄 Открыть заново», бейдж `✓ завершена`, прогресс «N/M подзадач», зачёркнутые/зелёные выполненные элементы. Выполненные подзадачи также скрываются из «Дедлайны на сегодня».
- **LLM-контекст:** для Q&A в контекст каждой кампании теперь добавляются её статус и прогресс `done/total`, а у подзадач — отметка `[✓ выполнено] / [ ] не выполнено`. Это позволяет отвечать на вопросы вида «что ещё осталось по X?», «сколько кампаний завершено?».

7. Память диалога (новое)

- **Хранение:** таблица `chat_messages`, ключ диалога — имя пользователя из логина (или `_anonymous`). Сохраняются и пользовательские реплики, и ответы бота (включая текст подсказки в `other`, ответ Q&A, краткое описание сформированной кампании, реплики `clarify`).
- **Слой бизнес-логики** (`core/service.py`):
 - `save_chat_message(user_name, role, content)` — сохраняет одну реплику.
 - `load_chat_history(user_name, limit)` — последние `limit` сообщений в хронологическом порядке (по умолчанию 50 для UI, 10 для LLM-контекста: `CHAT_HISTORY_FOR_LLM = 10`).
 - `clear_chat_history(user_name)` — удаляет всю историю диалога этого пользователя.
- **LLM:** `answer_question` принимает параметр `history` и собирает `messages = [system + история + текущий вопрос]`. Текущая реплика пользователя исключается из истории, чтобы не дублироваться. Это даёт корректную работу с указательными местоимениями («там», «у этой», «а кто ответственный?»).
- **UI:** при логине история подгружается с бэкенда (`GET /api/chat/history?user_name=...`) и вставляется в окно чата вместо стартовой подсказки. В шапке есть ссылка «очистить

чат» — после подтверждения дергает `DELETE /api/chat/history` и перерисовывает окно.

8. Интерфейс: статический веб-чат

- **Реализация:** один файл `web/static/index.html` (разметка + стили + JS, без npm и сборки) — бэкенд отдаёт его на `GET /`.
- **Возможности:**
 - Чат с ботом: ввод сообщения → вызов `/api/chat`; превью кампании с кнопками «Подтвердить»/«Отменить»; режим уточнения (дослать недостающие данные сообщением → `/api/campaigns/refine`).
 - Восстановление и очистка переписки через `/api/chat/history`.
 - Боковая панель: список кампаний с подзадачами и чекбоксами выполнения, бейджи статуса, прогресс, список шаблонов, подзадачи с дедлайном сегодня.
 - Логин: имя + роль сохраняются в браузере (без паролей), передаются в запросы и попадают в реестр участников (`/api/users`) и в поле `created_by` кампании.
 - Формы добавления и редактирования шаблонов кампаний.

9. Сценарий обработки сообщения

```
сообщение пользователя
-> save_chat_message(user)
-> classify_message:
  "question" -> load_chat_history -> answer_question(контекст + история) -> тек
  "other"      -> подсказка о возможностях
  "campaign" -> parse_campaign -> enrich_parsed -> create_pending
                  не хватает полей? -> ответ "clarify" (список missing)
                                      пользователь дослывает -> /api/campaigns/r
                                      -> merge_parsed ->
                  всё есть?          -> ответ "campaign" (preview, pending_id)
                                      -> /api/campaigns/confirm
                                      -> pick_template -> детерминирован
                                      \-> либо decompo
                                      -> запись Campaign + Subtasks -> d
-> save_chat_message(assistant)
```

10. Сбор и запуск системы

- **Необходимое ПО:** Python 3.10+ (Docker-образ — `python:3.12-slim`); доступ в интернет к OpenRouter.

- **Зависимости** (`requirements.txt`): `fastapi`, `uvicorn[standard]`, `aiohttp`, `sqlalchemy`, `aiosqlite`, `python-dotenv`, `dateparser`.
- **Переменные окружения** (`.env`):

```
OPENROUTER_API_KEY=sk-or-v1-...
DEBUG=True
# опционально:
# HOST=0.0.0.0
# PORT=8000
```

- **Запуск локально:**

```
pip install -r requirements.txt
python main.py # или: uvicorn web.app:app --reload
# открыть http://127.0.0.1:8000/ (документация API: /docs)
```

- **Запуск в Docker:**

```
docker build -t marketing-service .
docker run -p 8000:8000 -e OPENROUTER_API_KEY=sk-or-v1-... \
-v "$PWD/marketing_bot.db:/app/marketing_bot.db" marketing-service
```

11. Деплой на Render

- В репозитории есть `render.yaml` (blueprint) и `Dockerfile` — бесплатный web-сервис на Docker, health-check `/api/health`, `autoDeploy`.
- Порядок: запустить репозиторий на GitHub → на render.com *New + Blueprint* → выбрать репозиторий → *Apply* → в созданном сервисе *Environment* добавить `OPENROUTER_API_KEY` → *Save* (передеплой).
- Ограничения бесплатного тарифа: сервис «засыпает» после ~15 мин простоя (первый запрос ~30–60 с); файловая система эфемерная — SQLite-файл сбрасывается при редеплое (для постоянных данных нужна внешняя БД через `DATABASE_URL`); `PORT` Render задаёт сам, приложение его подхватывает.

12. Обоснование выбора технологий

Требование	Реализация
Простой и быстрый MVP	Монолит, одна встроенная БД (SQLite), статический фронтенд без сборки, минимум зависимостей.

Понимание свободного текста на русском	LLM <code>openai/gpt-5-nano</code> через OpenRouter; промпты на русском, формат «только JSON»; нормализация дат через <code>dateparser</code> .
Предсказуемое планирование кампаний	Детерминированное разбиение по шаблонам (формулы на % бюджета и доле срока), LLM — только когда шаблон не подходит.
Контекстные ответы по кампаниям	В Q&A-промпт подмешивается актуальное состояние всех кампаний (включая прогресс и статусы) и последние реплики диалога — модель видит и факты, и тред переписки.
Отслеживание прогресса	Поля <code>completed_at</code> + авто-пересчёт статуса кампании по подзадам; UI с чекбоксами и бейджами.
Память диалога	Отдельная таблица <code>chat_messages</code> , history-aware <code>answer_question</code> , восстановление переписки в UI при логине.
Асинхронность и масштабируемость кода	FastAPI + Uvicorn, async SQLAlchemy / <code>aiosqlite</code> , async HTTP-клиент <code>aiohttp</code> .
Лёгкий деплой	Dockerfile + <code>render.yaml</code> , health-check, переменные окружения, бесплатный тариф Render.
Без привязки к Telegram	Бизнес-логика вынесена в <code>core/service.py</code> ; веб-слой и (legacy) Telegram-бот — лишь транспорт.

13. Известные ограничения и компромиссы

- **Зависимость от внешнего LLM:** без сети/ключа OpenRouter ключевые сценарии (разбор кампании, Q&A) недоступны; на ошибку — HTTP 502.
- **SQLite:** рассчитан на десятки–сотни записей и один процесс; для нагрузки/нескольких воркеров нужна внешняя СУБД.
- **Эфемерное хранилище на Render:** данные сбрасываются при редеплое (бесплатный тариф) — включая историю диалога.
- **Аутентификации нет:** «логин» — это только имя+роль в браузере; данные общие для всех пользователей, история диалога разделена лишь по имени.
- **Качество разбора** зависит от формулировки сообщения и поведения модели; есть страховки (уточняющий диалог, дефолтная одна подзадача), но не гарантируют идеальный результат.
- **Напоминания** реализованы как запрос «что сделать сегодня» (`/api/reminders/today`), а не как push — нет планировщика рассылок.
- **Память диалога — без summarization:** в контекст LLM подмешиваются только последние 10 реплик. Для длинных диалогов более старый контекст забывается.

14. Возможные улучшения (без смены стека)

- Вынести `DATABASE_URL` в окружение и перейти на PostgreSQL для постоянного хранения и масштабирования.
- Добавить аутентификацию пользователей и разграничение доступа к кампаниям и истории диалога.
- Кэшировать ответы LLM (например, классификацию и Q&A по неизменному контексту).
- Стриминг ответа LLM в веб-чат (Server-Sent Events / WebSocket).
- Полноценный планировщик напоминаний (фоновая задача + уведомления) с учётом отметок выполнения.
- Сжатие старой истории диалога через LLM-summarization при росте переписки.
- Тесты на бизнес-логику разбиения, слияния pending-кампаний и авто-пересчёта статусов.